

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent work on deep learning optimization has proposed several first-order algorithms, including Schedule-Free optimizers, AdEMAMix, MARS and Lion, that modify the conventional construction of momentum-based update directions. A separate theoretical literature has established that stochastic gradient descent (SGD) can be accelerated in noise-dominated regimes when the momentum-retention coefficient is decoupled from the coefficient assigned to the current stochastic gradient. This paper formalizes the correspondence between these two developments. We derive update-level equivalences and reparameterizations connecting recently proposed optimizer recursions to accelerated SGD variants, and we evaluate the resulting interpretation through preliminary experiments on a 150m-parameter language modeling task. The experimental evidence indicates that AdEMAMix, whose update is the closest instantiation of the accelerated-SGD template among the considered methods, obtains the strongest performance. Motivated by this correspondence, we introduce Simplified-AdEMAMix, a modification of AdEMAMix that preserves its empirical behavior in both small- and large-batch regimes while eliminating the need to maintain two separate momentum states. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent deep learning optimization work has introduced algorithms such as Lion (Chen et al., 2023), Schedule-FreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these methods arise from distinct design criteria, they share a common structural modification: each changes the mechanism by which the optimizer accumulates momentum information and injects it into the parameter update.

A separate theoretical literature studies acceleration for gradient methods in the presence of stochastic gradient noise. Classical momentum mechanisms, including heavy-ball and Nesterov momentum, yield acceleration for deterministic gradient descent in settings such as quadratic objectives, but they do not imply the analogous acceleration guarantee for SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation has motivated alternative momentum parameterizations that are explicitly designed for noisy stochastic optimization (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Across these accelerated SGD variants, the coefficient governing persistence of the historical momentum state is not tied to the coefficient multiplying the current stochastic gradient in the update direction.

The central objective of this paper is to specify the mapping between these two research directions at the level of optimizer recursions. We show that Schedule-Free SGD is mathematically equivalent to an accelerated SGD update followed by a weight-averaging operation. We further show that Lion, Schedule-Free AdamW, and AdEMAMix can be interpreted as accelerated-SGD-style updates augmented with preconditioning or coordinate-wise transformations. Although prior work has identified related momentum-averaging relationships (Defazio, 2021), the explicit correspondence between these recently proposed optimizers and accelerated SGD variants has not, to our knowledge, been formalized in this update-level form.

We support the formal analysis with experiments on a 150m-parameter decoder-only transformer trained on 15b tokens with a 32k-token batch size, which is intended to instantiate a noise-dominated training regime. Consistent with the derived correspondence, Schedule-Free AdamW performs similarly to accelerated SGD-based AdamW (Algorithm 3), and accelerated variants yield modest improvements in the small-batch setting. We also evaluate the same optimizer families at larger batch sizes and observe that these gains diminish when gradient noise is reduced, as predicted by the accelerated-SGD interpretation.

The paper makes the following contributions:

1. We derive explicit theoretical correspondences between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.
2. We empirically evaluate AdamW, Schedule-Free AdamW, AdEMAMix, and MARS on a 150-million-parameter decoder-only transformer, and the results show that AdEMAMix, whose recursion has the closest structural alignment with accelerated SGD variants, obtains the best performance among these methods.
3. We show that, relative to Adam, the gains associated with these accelerated-SGD-like update structures decrease at large batch sizes; in this regime, Adam with momentum scheduling can match the performance of AdEMAMix.
4. We find that Schedule-Free AdamW underperforms AdamW with cosine decay at high batch sizes, and we attribute this behavior to the intrinsic coupling between the momentum coefficient and the weight-averaging coefficient in Schedule-Free optimizers.
5. We propose Simplified-AdEMAMix, a modification of AdEMAMix that preserves AdEMAMix-level performance across both small- and large-batch regimes while requiring only a single momentum state.

2 RELATED WORK

We summarize prior work on accelerated SGD variants and on optimizer constructions whose update rules are technically relevant to the correspondences analyzed in this work.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares objectives. Kidambi et al. (2018) subsequently simplified the corresponding update rule and proved that standard momentum does not yield acceleration in this setting. Later analyses (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended the accelerated-SGD framework to general convex and strongly convex objectives under different assumptions on smoothness, noise, and problem structure.

Several practical optimizers also contain update structures that resemble the accelerated SGD variants described above. Lucas et al. (2019) proposed an update direction defined as a weighted sum of multiple momentum terms with distinct coefficients. Ma & Yarats (2019) developed an optimizer explicitly motivated by the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) introduced an optimizer discovered by genetic search; its update assigns separate coefficients to the instantaneous gradient and to the momentum state, matching the coefficient separation present in accelerated SGD parameterizations. Pagliardini et al. (2024) similarly constructs its parameter update from two momentum scales and combines them linearly in the final step direction.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a standard mechanism for modifying the convergence dynamics of gradient descent in deterministic optimization. For parameters w_t and momentum coefficient β , the conventional momentum recursion is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is commonly used in stochastic optimization to reduce variance in the returned iterate. Rather than selecting only the terminal iterate w_T , the procedure maintains a weighted iterate average $\bar{w}_T$, with averaging coefficients denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, the weight-averaging procedures use coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for a constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We use the following generalized update template to represent a broad class of accelerated SGD methods:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ denote possibly time-dependent scalar coefficients, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ marks coefficients associated with this accelerated SGD parameterization.

Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum. Conversely, many accelerated SGD algorithms in the literature, including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023), can be represented by the same template after algebraic reparameterization. Appendix B provides the corresponding reductions.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section derives explicit update-level connections between accelerated SGD and existing optimizers, including Schedule-Free methods and AdEMAMix. The same analysis motivates a simplified AdEMAMix variant that uses one momentum state while preserving AdEMAMix-level performance in both small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer designed to remove the need for an explicit learning-rate schedule. Using the notation of Defazio et al. (2024), its update equations are:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

In this formulation, y_t denotes the parameter vector at which the stochastic gradient is evaluated, whereas x_t denotes the parameter vector used for model evaluation.

We first rewrite the dynamics in terms of y_t and an auxiliary momentum variable m_t , defined as follows:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Algebraic simplification of this definition yields:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Therefore, m_t satisfies the momentum recursion in Equation (1) with $\beta_{a,t} = 1 - c_t$. We next express the induced update on y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus, with $w_t = y_t$, the y_t recursion has the accelerated SGD form in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$. The training weights in Schedule-Free SGD therefore follow an accelerated SGD update. Because Schedule-Free SGD evaluates the averaged weights x_t , we additionally characterize the dynamics of x_t :

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

This expression identifies x_t as a weighted average of the y_t iterates. Recursively expanding the recurrence shows that, when c_t is constant, x_t is an exponential average of y_t . Hence Schedule-Free SGD can be represented as an accelerated SGD trajectory followed by a weight-averaging map.

The resulting behavior of Schedule-Free SGD can therefore be decomposed into two algorithmic components:

1. Improved performance relative to standard momentum SGD, induced by the accelerated SGD component.
2. Compatibility with a constant learning rate, induced by the weight-averaging component (specifically, tailed weight averaging; see Section 4.1.3).

This construction also gives Schedule-Free SGD/Adam two implementation-level advantages:

- It does not allocate an additional buffer solely for weight averaging.
- It avoids exposing a separate weight-averaging coefficient as a hyperparameter.

However, Section 5.1 shows that coupling the momentum coefficient to the weight-averaging coefficient becomes empirically problematic in the large-batch regime.

4.1.1 CASE: $\beta = 0.0$

As observed in (Defazio et al., 2024), setting $\beta = 0$ reduces Schedule-Free SGD to standard SGD with weight averaging. Since $c_t = 1/t$, the averaging operation is active from the initial iteration.

4.1.2 CASE: $\beta = 1.0$

As observed in (Defazio et al., 2024), setting $\beta = 1$ reduces Schedule-Free SGD to standard momentum SGD, with the effective momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD choice $\beta = 0.9$, the parameterization implies the following coefficient behavior:

- Because c_t scales as $1/t$, the effective momentum coefficient scales as $1 - 1/t$.
- The ratio between the coefficient on the current gradient and the coefficient on momentum is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- The averaging window corresponds approximately to the most recent 10% of the iterates.

4.2 LION

Lion (Chen et al., 2023) applies the following update rule:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) can therefore be interpreted as constructing an accelerated-SGD-style direction and then applying a coordinate-wise sign transformation before the parameter update.

4.3 MARS

We next show that the practical version of MARS (Yuan et al., 2024), denoted MARS-Approx, can be expressed as an accelerated SGD update combined with a preconditioning operation. Omitting bias correction and clipping, the update equations are:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t denote first- and second-order moment states, respectively, and x_t denotes the model parameters. Introducing $\hat{m}_t = m_t - \gamma g_t$ yields the equivalent representation:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This representation shows that the momentum recursion has the same coefficient structure as the accelerated SGD template. MARS differs from AdamW, however, in the construction of its preconditioner, and we evaluate the empirical consequences of this distinction in Section 5.

4.4 ADEMAMIX

AdEMAMix (Pagliardini et al., 2024) has a close structural relationship to accelerated SGD-based AdamW. Instead of forming a linear combination of the current gradient and a single momentum state, as in the accelerated SGD template, AdEMAMix maintains two momentum states with different decay coefficients and uses a linear combination of those states in the update direction. Algorithm 1 gives the corresponding algorithmic step.

For analytical clarity, we consider the AdEMAMix specialization with $\beta_1 = 0$. Pagliardini et al. (2024) report that this specialization is nearly performance-equivalent to the full method at small batch sizes, and our experiments in Section 5 reproduce the same behavior. Under $\beta_1 = 0$, the AdEMAMix update aligns with the general accelerated SGD form in Equation (1). Moreover, the schedules prescribed for β_3 , which acts as the momentum coefficient, and α , which controls the relative contribution of the slow momentum term, closely match theoretical schedules used in accelerated SGD (Gupta et al., 2023).

For smooth convex objectives, stochastic acceleration requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for a constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately implements this schedule through its increasing β_3 parameterization.

In the accelerated SGD template, the momentum state is maintained as:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas the corresponding AdEMAMix momentum state in Algorithm 1 is updated according to:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the cumulative contribution of historical gradients in the accelerated SGD momentum state m_t grows proportionally to t . The AdEMAMix coefficient α follows an analogous proportional scaling. This structural correspondence is consistent with the improved empirical performance of AdEMAMix relative to the other optimizers in Figure 1.

In the large-batch regime, however, AdEMAMix degrades when $\beta_1 = 0.0$, as also reported by Pagliardini et al. (2024). The analysis of Gupta et al. (2023) suggests that the coefficient on the current gradient should decrease as batch size increases. By contrast, AdEMAMix assigns a fixed coefficient of 1 to the current gradient, which provides a plausible explanation for its reduced large-batch performance under $\beta_1 = 0.0$.

The next subsection defines a simplified AdEMAMix variant that explicitly includes a coefficient on the current gradient, removes the need to schedule α , and maintains only one momentum state. We then empirically verify that this simplified optimizer matches AdEMAMix in both small- and large-batch settings.

4.5 SIMPLIFIED ADEMAMIX

Motivated by the preceding correspondence, we define a simplified optimizer that removes the second momentum buffer and eliminates scheduling of α . Algorithm 2 specifies the update, using theory-style momentum rather than the normalized exponential-moving-average convention. The final update assigns a fixed coefficient α to the instantaneous gradient. Setting $\alpha = 0$ recovers standard Adam after the appropriate reparameterization of η and β_1 . The experiments in Section 5 show that this simplified variant matches AdEMAMix across both small and large batch sizes.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

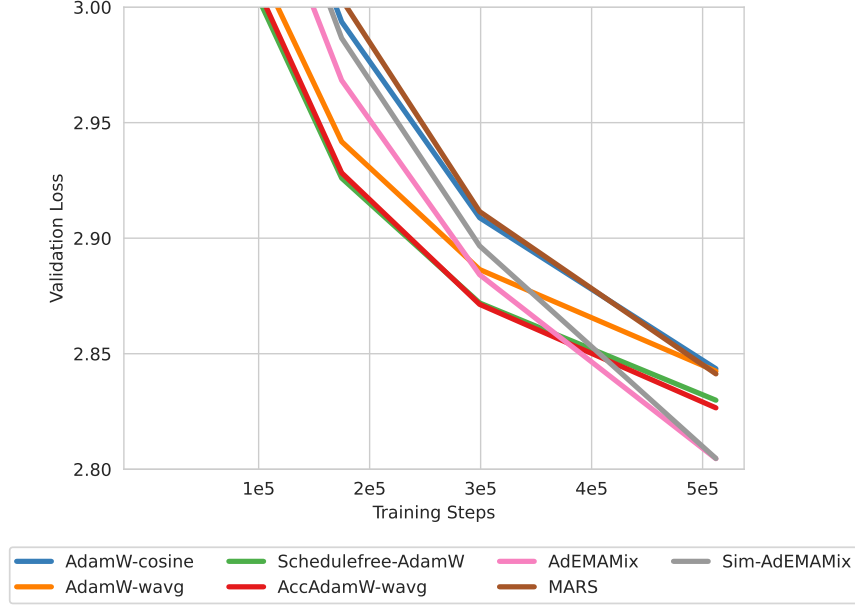


Figure 1: Best-run validation comparison for the optimizers evaluated in Section 5 on a language modeling task using a decoder-only 150m-parameter transformer. AdEMAMix and simplified-AdEMAMix obtain the strongest validation performance, consistent with their close update-level correspondence to accelerated SGD variants.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

5 EXPERIMENTS

This section reports experiments on a 150-million-parameter decoder-only transformer trained for language modeling on the C4 dataset. We use sequence length 1024 and batch size 32, and train for 15 billion tokens ($\approx 5 \times$ Chinchilla). This experimental configuration is intended to place the optimization process in a noise-dominated regime.

The evaluation includes the following optimizer configurations:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix

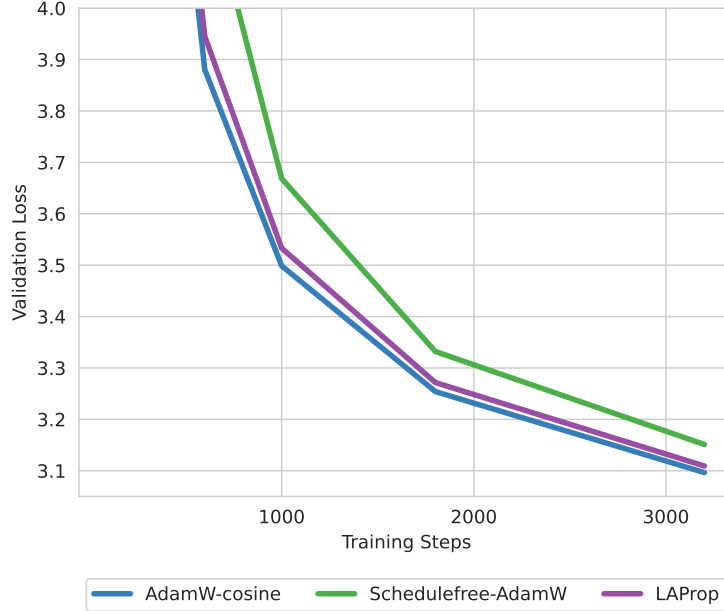


Figure 2: Best-run validation comparison of AdamW with cosine decay, Schedule-Free AdamW, and LARProp in the higher-batch-size setting. Section 5.1 specifies the corresponding experimental configuration.

7. Simplified-AdEMAMix

Section A specifies the hyperparameter sweep ranges used for these configurations.

As shown in Figure 1, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging obtain comparable validation performance, supporting the derived equivalence between the two update structures. Both methods outperform AdamW with cosine decay and AdamW with tailed weight averaging in this setting. AdEMAMix and Simplified-AdEMAMix further improve on all evaluated methods; we attribute this behavior to their closer structural alignment with accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

The preceding experiments isolate the small-batch regime, where stochastic gradients are comparatively noisy. We next evaluate the same optimizer families in a large-batch regime to determine whether the observed advantages persist when gradient noise is reduced. In this setting, models are trained with a batch size of 1 million tokens for 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: Figure 2 shows that Schedule-Free AdamW substantially underperforms AdamW in the large-batch setting. We attribute this gap to the coupling between the weight-averaging coefficient and the momentum coefficient. At high batch sizes, the optimal momentum value is substantially below $1 - 1/t$. Although a scaled schedule of the form $\approx 1 - r/t$ with $r \geq 1$ can reduce the effective momentum, increasing r also shortens the effective weight-averaging window.

A second algorithmic distinction concerns the ordering of momentum and preconditioning. AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, making it structurally similar to LARProp (Ziyin et al., 2021). However, Figure 2 shows that AdamW and LARProp achieve comparable performance, suggesting that this ordering difference is not the principal cause of the observed performance gap.

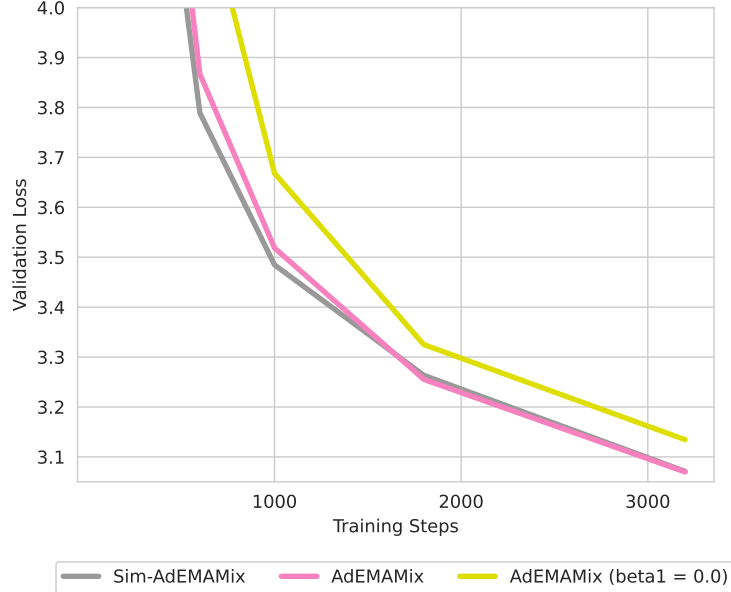


Figure 3: Best-run validation comparison of AdEMAMix, AdEMAMix with $\beta_1 = 0.0$, and the proposed simplified AdEMAMix variant in the higher-batch-size experiments. Section 5.1 specifies the corresponding experimental configuration.

AdEMAMix: In the large-batch regime, consistent with Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix produces a substantial degradation relative to the full two-momentum version. This behavior is consistent with the accelerated-SGD interpretation: AdEMAMix assigns a fixed coefficient of 1 to the current gradient, whereas theoretical accelerated SGD variants (Gupta et al., 2023) require the current-gradient coefficient to decrease as batch size increases.

Figure 3 also shows that the proposed Simplified-AdEMAMix variant matches AdEMAMix while removing the second momentum state. This equivalence is obtained at $\alpha = 0.0$, in which case Simplified-AdEMAMix reduces to standard Adam with momentum scheduling.

6 CONCLUSION

This work formalizes update-level connections between accelerated SGD variants and several recently proposed optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. The empirical results support the analysis: in the small-batch regime, AdEMAMix, whose recursion most closely matches the accelerated-SGD structure, obtains the strongest performance among the evaluated methods.

Using this correspondence, we define Simplified-AdEMAMix, an optimizer that removes the second momentum buffer while retaining the relevant accelerated-SGD-style coefficient structure. Experiments show that Simplified-AdEMAMix matches AdEMAMix in both small- and large-batch settings while eliminating the additional memory overhead associated with two maintained momentum states.

REFERENCES

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, De-*

- ember 10 - 16, 2023, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

The small-batch experiments use the following hyperparameter grids:

1. AdamW with cosine decay and 51.2k warmup: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The selected values were $\beta_1 = .9$ and $\beta_2 = .999$, matching the default configuration. Although larger-batch training often uses $\beta_2 = .95$, the benefit of larger β_2 values at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay and 10k warmup: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, fixing β_1, β_2 to the selected values from the preceding sweep. This configuration performed worse than the 51.2k-step warmup.
3. AdamW with constant-fraction weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD-based AdamW with cosine decay: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, and $\beta_3 = 0.9$.
6. Accelerated SGD-based AdamW with constant learning rate and weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, and δ in $[0.05, 0.1]$.
7. Accelerated SGD-based AdamW with cosine decay and weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, and $\beta_3 = 0.9$.
8. Schedule-Free AdamW with constant learning rate: learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, and $\beta_2 = 0.999$.
9. Schedule-Free AdamW with cosine decay: learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, and $\beta_2 = 0.999$.
10. MARS: learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, and γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, with precondition 1d set to True.
11. AdEMAMix: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, and α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix: learning rate in $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, and α in $[10, 20, 50, 100]$.

The large-batch experiments use the following hyperparameter grids:

1. Schedule-Free AdamW: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, and r in $[0.0, 5.0, 9.0, 50.0]$.
2. AdamW: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.9, 0.95]$.
3. LProp: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.9, 0.95]$.
4. AdEMAMix: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, and α in $[2, 4, 8, 16]$.
5. Sim-AdEMAMix: learning rate in $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, and α in $[0.0, 0.5, 1.0]$.

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Equation (1) defines the general accelerated SGD template. This appendix verifies that the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) can each be expressed within that template after algebraic reparameterization.

B.1 AGNES

The update rule of Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n , while the final objective value is evaluated at x_n . These equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} obeys an update equation of the form specified in Equation (1).

B.2 ASGD

The update rule of Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} , and the objective is evaluated on the tail-averaged x iterates.

The same update can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

These rewritten equations instantiate the template in Equation (1).

B.3 MASS

The update rule of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t , and the objective is evaluated at w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The resulting equations again match the form in Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update rule of Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

This system can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$